

// Jivops

HAProxy

Guía completa — 20 lecciones

Fundamentos · Health checks y ACLs · Seguridad y rendimiento · Operación y producción

Contenido

Módulo 1: Fundamentos

- Lección 01 — Qué es HAProxy
- Lección 02 — Estructura del fichero de configuración
- Lección 03 — Frontends y bind
- Lección 04 — Backends y algoritmos de balanceo
- Lección 05 — Modos tcp y http

Módulo 2: Health checks y ACLs

- Lección 06 — Health checks
- Lección 07 — ACLs y use_backend
- Lección 08 — Persistencia de sesión
- Lección 09 — Cabeceras y redirecciones
- Lección 10 — Timeouts

Módulo 3: Seguridad y rendimiento

- Lección 11 — Terminación TLS
- Lección 12 — Rate limiting con stick tables
- Lección 13 — Control de acceso y protección
- Lección 14 — Rendimiento: threads, maxconn y keep-alive
- Lección 15 — API de runtime y socket de administración

Módulo 4: Operación y producción

- Lección 16 — Estadísticas y monitorización
- Lección 17 — Alta disponibilidad con keepalived
- Lección 18 — Recargas sin cortes
- Lección 19 — Logs y troubleshooting
- Lección 20 — Proyecto final: HAProxy listo para producción

Módulo 1: Fundamentos

Lección 01 — Qué es HAProxy

Un balanceador de carga y proxy inverso de código abierto, de alto rendimiento, que reparte conexiones TCP (capa 4) o peticiones HTTP (capa 7) entre servidores de backend, con comprobaciones de salud, terminación TLS y reglas flexibles.

En modo tcp reenvía conexiones sin interpretarlas; en modo http entiende las peticiones, por lo que puede decidir por ruta, host o cabeceras.

Lección 02 — Estructura del fichero de configuración

En secciones: global (parámetros del proceso), defaults (valores comunes), y frontend, backend o listen para definir por dónde entra el tráfico y a qué servidores se envía.

Para definir valores comunes (modo, timeouts, opciones) que heredan las secciones siguientes, evitando repetirlos.

Lección 03 — Frontends y bind

El punto de entrada del tráfico: con bind indica en qué direcciones y puertos escucha (y si usa TLS), y con reglas (ACLs, use_backend) decide a qué backend enviar cada conexión o petición.

Define el backend al que se envía el tráfico cuando ninguna regla use_backend coincide.

Lección 04 — Backends y algoritmos de balanceo

Entre otros, roundrobin (reparto por turnos, el predeterminado), leastconn (envía al servidor con menos conexiones activas, útil con conexiones largas) y source (mismo cliente al mismo servidor según el hash de su IP).

Con conexiones largas o tiempos de respuesta desiguales, porque envía al servidor con menos conexiones activas.

Lección 05 — Modos tcp y http

HAProxy reenvía los bytes de la conexión sin interpretar el protocolo, por lo que sirve para bases de datos, SMTP o TLS sin descifrar, pero no puede usar reglas por cabeceras o rutas HTTP.

Porque PostgreSQL usa su propio protocolo, no HTTP; en modo tcp HAProxy reenvía las conexiones sin interpretarlas.

Módulo 2: Health checks y ACLs

Lección 06 — Health checks

Con la opción `check` en cada línea `server`, que prueba la conexión TCP de forma periódica (`inter`), y con `option httpchk` para comprobar una URL HTTP; un servidor se marca caído tras `fall` fallos y vuelve tras `rise` éxitos.

Porque un puerto abierto no garantiza que la aplicación funcione; la URL de salud comprueba que responde correctamente.

Lección 07 — ACLs y `use_backend`

Para definir condiciones sobre el tráfico (`host`, `ruta`, `cabecera`, `IP de origen`, etc.) y usarlas en reglas como `use_backend`, `http-request deny` o `redirect`, de modo que se enrute o filtre según lo que contiene la petición.

Defines una ACL con `path_beg /api`, usas `use_backend api_servers` if esa ACL y dejas `default_backend` para el resto.

Lección 08 — Persistencia de sesión

Con una cookie de persistencia insertada por HAProxy (`cookie SRVID insert`) o con una `stick table` asociada al origen, de modo que las peticiones del mismo cliente se dirijan al servidor que atendió la primera.

Puede desequilibrar la carga y, si el servidor cae, el usuario pierde la sesión; es mejor guardar el estado fuera de los servidores.

Lección 09 — Cabeceras y redirecciones

Con una regla en el frontend del puerto 80, por ejemplo `http-request redirect scheme https unless { ssl_fc }`, que envía una redirección a la versión segura; además `option forwardfor` añade la IP real del cliente en `X-Forwarded-For`.

Porque sin ella solo vería la IP del proxy y no la del cliente real, lo que afecta a logs y controles por IP.

Lección 10 — Timeouts

`timeout connect` (tiempo para conectar con un servidor), `timeout client` (inactividad del cliente) y `timeout server` (inactividad del servidor); sin ellos HAProxy emite avisos y las conexiones lentas pueden acumularse.

Se cortan las respuestas lentas o largas y los clientes reciben errores 504, aunque el servidor siga procesando.

Módulo 3: Seguridad y rendimiento

Lección 11 — Terminación TLS

Con `bind *:443 ssl crt /ruta/cert.pem` el frontend descifra el tráfico, de modo que puede aplicar reglas HTTP y reenviar a los backends por HTTP o re-cifrando; se configuran versiones y cifrados modernos y puede negociar HTTP/2 con ALPN.

Cuando se exige cifrado de extremo a extremo y HAProxy no debe descifrar; se usa modo `tcp` y se pierden las reglas por HTTP.

Lección 12 — Rate limiting con stick tables

Declarando una stick table que almacene `http_req_rate` por IP, registrando cada petición con `http-request track-sc0 src` y denegando con `deny_status 429` cuando la tasa supera el umbral.

La tasa de peticiones por IP, con una entrada que expira tras un tiempo, para decidir si se deniega al superar el umbral.

Lección 13 — Control de acceso y protección

Denegar o limitar por IP con ACLs, bloquear rutas sensibles (como `/admin`) fuera de redes de confianza, limitar conexiones y tamaño de cabeceras, y usar timeouts cortos frente a clientes lentos.

Con una ACL de ruta (`path_beg /admin`) y otra de origen (`src` de la red interna), denegando si es `/admin` y no viene de esa red.

Lección 14 — Rendimiento: threads, maxconn y keep-alive

El número de hilos (`nbthread`), los límites de conexiones (`maxconn` global y por servidor), el uso de `keep-alive` hacia clientes y servidores y los parámetros del sistema operativo como los límites de descriptores de fichero.

Para no saturar el servidor: las conexiones adicionales esperan en cola en HAProxy en lugar de llegar todas al backend.

Lección 15 — API de runtime y socket de administración

Consultar y cambiar el estado en caliente sin recargar, por ejemplo poner un servidor en `drain` o en `mantenimiento` (`set server backend/srv state drain`), cambiar pesos o ver estadísticas, normalmente con `socat` sobre el stats socket.

Porque no llegan conexiones nuevas y las actuales terminan, así se reinicia sin cortar a los usuarios.

Módulo 4: Operación y producción

Lección 16 — Estadísticas y monitorización

Con la página de estadísticas (stats uri), que muestra el estado de frontends, backends y servidores, y con métricas Prometheus mediante su exportador integrado; conviene alertar sobre servidores caídos, errores 5xx y colas de conexiones.

Servidores de backend caídos y aumento de errores 5xx, o colas de conexiones crecientes.

Lección 17 — Alta disponibilidad con keepalived

Con dos o más instancias y una IP virtual gestionada por keepalived (VRRP): si el nodo activo falla, la IP flotante pasa al nodo de respaldo, y ambos mantienen la misma configuración.

Que el proceso de HAProxy funcione, mediante un script de comprobación; si no, la IP podría seguir en un nodo con el balanceador caído.

Lección 18 — Recargas sin cortes

Con una recarga (reload) en modo master-worker: se lanza un proceso nuevo con la configuración nueva, que acepta conexiones nuevas, y el antiguo termina las que tenía abiertas; antes conviene validar con haproxy -c -f.

Para evitar que una configuración inválida impida la recarga o rompa el servicio en producción.

Lección 19 — Logs y troubleshooting

Revisando los logs en formato httplog (con el estado de terminación y los tiempos), el estado de los servidores en stats, los health checks, los timeouts y la conectividad hacia el backend.

Indica quién cerró la conexión y en qué fase (cliente, servidor o timeout), lo que orienta al origen del problema.

Lección 20 — Proyecto final: HAProxy listo para producción

Dos nodos HAProxy con keepalived, TLS moderno con redirección a HTTPS, health checks a URLs de salud, timeouts y maxconn definidos, rate limiting y ACLs de protección, recarga validada con haproxy -c y monitorización con alertas.

Alta disponibilidad con IP virtual, health checks de aplicación y monitorización con alertas (más TLS, timeouts y límites).